NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Level: Bachelor • Programme: BE • Year: 2026

Course: .NET Technologies (Elective) • Full Marks: 100 • Time: 3 hrs

.NET Technologies Model Answers

Simple Language Edition

All questions with every **OR** alternative | Easy words, diagrams and tables

Covered: Q1–Q7 (17 answers in total)

Architecture • Page Life Cycle • Connection Strings • appsettings.json • EF Core

Exception Handling • LINQ • async/await • SOLID • State Management

Dependency Injection • RESTful Web APIs • Microservices • GC • Security • OOP

PREPARED BY

Arpan Adhikari

Contents

Question	Topic	Marks
1 (a)	Architecture of the .NET platform and its key components	7
1 (b)	.NET Framework vs .NET Core vs unified .NET	8
2 (a)	ASP.NET Page Life Cycle and its events	7
2 (b)	Connection string and its parameters	8
3 (a)	Role and structure of appsettings.json	7
3 (b)	Role and advantages of Entity Framework Core	8
4 (a)	Exception handling in C# — try, catch, finally	7
4 (b)	Language Integrated Query (LINQ)	8
5 (a)	Asynchronous programming with async and await	7
5 (b)	SOLID principles and their importance	8
5 (b) OR	State management in ASP.NET	8
6 (a)	Dependency injection and service lifetimes	7
6 (b)	RESTful Web APIs using ASP.NET Core	8
6 (b) OR	Benefits and challenges of microservices	8
7 (a)	Short note: Garbage Collection in .NET	5
7 (b)	Short note: Authentication and Authorization	5
7 (c)	Short note: Abstract Class vs Interface	5

How to write each answer. Follow the same four steps every time: **(1)** write the definition from the shaded box, **(2)** draw the diagram, **(3)** write the points with the bold keywords, **(4)** end with the conclusion line. Marks come mostly from the diagram and the keywords, not from long paragraphs.

Q1 (a). Illustrate the overall architecture of the .NET platform and explain the functions of its key components. 7 Marks

> **Definition.** The **.NET platform** is a framework made by Microsoft for building and running applications — desktop, web, mobile and cloud. You can write code in any .NET language. That code is first changed into a common language called **IL (Intermediate Language)**, and then the **CLR (Common Language Runtime)** runs it on top of the operating system.

The architecture is made of **layers**. Each layer talks only to the layer just below it. This is why .NET can support many languages and can run on many machines.

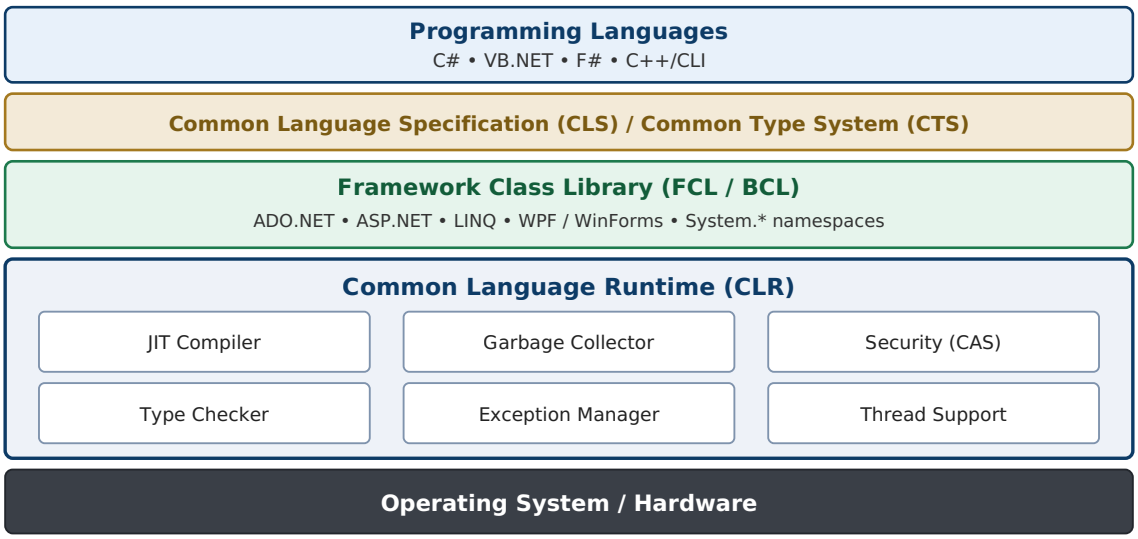

Fig 1.1: Layered architecture of the .NET platform

Functions of the Key Components

1. **CLR (Common Language Runtime)** — the engine that runs .NET programs. It loads the program, uses the **JIT compiler** to change IL into machine code, and takes care of **memory (Garbage Collection)**, **type safety**, **exceptions**, **threads** and **security**. Code that runs inside the CLR is called **managed code**.
2. **CTS (Common Type System)** — a common set of data types for all .NET languages, so that code written in one language can be used in another. Example: `int` in C# and `Integer` in VB.NET are both really `System.Int32`.
3. **CLS (Common Language Specification)** — a set of rules that every .NET language must follow. If all languages follow these rules, they can work together. This is called **language interoperability**.
4. **FCL / BCL (Framework Class Library)** — a big collection of ready-made classes for common work like file handling, collections, networking, XML and database access. They are grouped into **namespaces** such as `System`, `System.IO` and `System.Data`. So the programmer does not have to write everything from the beginning.
5. **Assemblies and Metadata** — after compiling, the code is stored in an **assembly** (a `.dll` or `.exe` file). The assembly contains the IL code, the **metadata** (information about the classes and methods inside it) and a **manifest** (name, version, and the list of other assemblies it needs). Because this information is inside the file itself, assemblies are called **self-describing**. There is no need to register them in the Windows registry — just copy the folder and the application works.

6. **Application Models** — the technologies built on top of the library that we actually use to make programs: **ASP.NET Core** for websites, **ADO.NET / EF Core** for databases, **WinForms / WPF** for desktop and **MAUI** for mobile.

How a .NET Program Runs (two-step compilation)

Fig 1.2: Two-step compilation — source → IL → native code

First the language compiler changes the source code into **IL code** and stores it in an assembly. Then, while the program is running, the **JIT compiler** changes that IL into machine code for that particular computer. Because of these two steps, the same file can run on different machines, and many languages can be used.

Advantages of this Architecture

- Many languages can be used together in one project.
- Memory is managed automatically, so fewer crashes and memory leaks.
- Code is checked before running, so it is more secure.
- Installing is easy because assemblies carry their own information.

Q1 (b). Compare .NET Framework, .NET Core and the unified .NET platform in terms of runtime, tooling and application development.

8 Marks

In simple words:

- **.NET Framework (2002)** — the old one. It runs only on Windows.
- **.NET Core (2016)** — a full rewrite that made .NET work on Windows, Linux and Mac.
- **Unified .NET (2020 onwards)** — Microsoft joined both into one platform, simply called **.NET** (now .NET 8 / 9). This is used for all new projects.

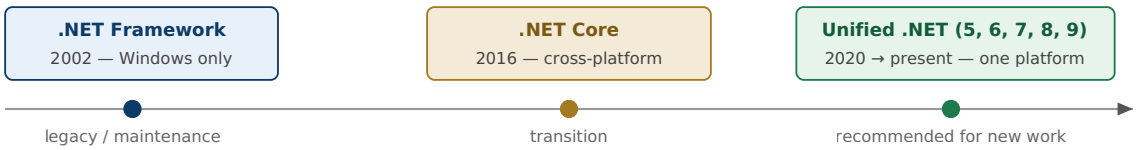

Fig 1.3: Evolution of the .NET platform

Comparison Table

Point	.NET Framework	.NET Core	Unified .NET (5+)
Started / status	2002; old, only bug fixes now (last is 4.8)	2016; stopped, merged into .NET 5	2020 to now; actively developed, new version every year
Where it runs	Only Windows	Windows, Linux, Mac	Windows, Linux, Mac, mobile and browser
Runtime	CLR, installed once for the whole computer	CoreCLR, light and modular; can be copied inside the app folder	CoreCLR with AOT options; very fast startup
Two versions together	Not possible — upgrading may break old apps	Possible — side by side	Possible — side by side
Library	One big library installed with Windows	Small parts downloaded as NuGet packages	One single library for everything
Tools	Only Visual Studio on Windows	`dotnet` command line, VS Code, any OS	Same tools plus hot reload and Docker support
What you can build	Web Forms, MVC 5, WCF, WinForms, WPF	ASP.NET Core (MVC and Web API together), console apps	ASP.NET Core, Blazor, Minimal APIs, gRPC, MAUI
Speed	Slow and heavy	Much faster	Fastest
Open source	No	Yes (free, MIT)	Yes (free, MIT)

1. Runtime

In .NET Framework, the runtime is installed **once for the whole computer**, so every application uses the same one. If it is upgraded, an old application may stop working. In .NET Core and unified .NET, an application can **carry its own runtime** inside its folder. So two applications can safely use two different versions at the same time. Unified .NET also adds **AOT compilation**, which makes the program start very quickly.

2. Tooling

.NET Framework needs **Visual Studio on Windows**. .NET Core gave us the **dotnet command line** (`dotnet new`, `dotnet build`, `dotnet run`), so code can be written in VS Code on Linux or Mac also. The project file also became short and clean. Unified .NET keeps all this and adds **hot reload** (see the change without restarting) and easy Docker containers.

3. Application Development

Old technologies like **Web Forms and WCF** work only in .NET Framework. .NET Core joined MVC and Web API into one model called **ASP.NET Core**. Unified .NET adds more: **Blazor** (write C# instead of JavaScript for the browser), **gRPC** (fast service calls) and **MAUI** (one project runs on Android, iOS, Windows and Mac).

> **Conclusion:** .NET Framework is old and Windows-only, .NET Core made .NET cross-platform and fast, and unified .NET joined both into one modern platform. So all new applications should be made in unified .NET.

Q2 (a). Describe the sequence of events in the ASP.NET Page Life Cycle. Explain the purpose of the major stages involved in processing and rendering a web page. 7 Marks

Definition. In ASP.NET Web Forms, when a user asks for an `.aspx` page, the server **creates the page, passes it through a fixed set of stages and events, makes the HTML, and then destroys the page**. This fixed order is called the **Page Life Cycle**. We must know it so that we write our code in the correct event.

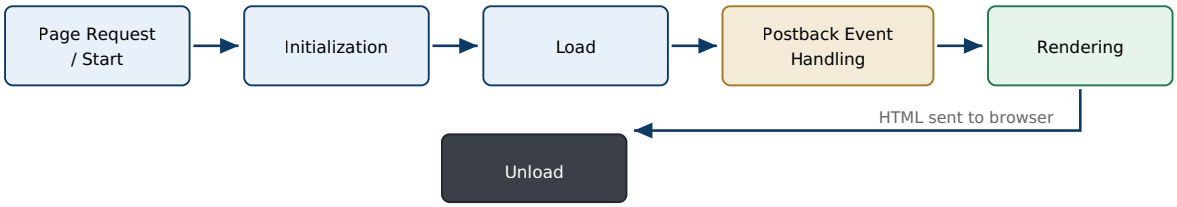

Fig 2.1: Stages of the ASP.NET Web Forms page life cycle

Purpose of the Major Stages

- **Page Request / Start:** the server decides whether to compile the page or use a saved copy, creates the page object and sets `IsPostBack`.
- **Initialization:** every control gets its ID and its properties. ViewState is **not** available here.
- **Load:** the old values of the controls come back from ViewState. Here we connect to the database and bind data.
- **Postback Event Handling:** the event caused by the user (button click, dropdown change) runs here, and validation also happens.
- **Rendering:** every control writes its HTML, and ViewState is saved into the page.
- **Unload:** the page is already sent to the browser. Now we close files and connections and free the memory.

Events in Order

#	Event	What it is used for
1	**PreInit**	First event. Set the master page and theme, and create **dynamic controls** here. ViewState is not ready yet.
2	**Init**	Each control is made ready. Child controls first, then the page. ViewState still not available.
3	**InitComplete**	Initialization is over. From here ViewState starts saving changes.
4	**PreLoad**	Runs after ViewState and posted data are put back into the controls.
5	**Load** (`Page_Load`)	**Most used event.** All controls are ready. Read values, get data from database and bind it. Use `if (!IsPostBack)` for first-time work only.
6	**Control events**	User events like `Button_Click` run here. Validation also runs here.
7	**LoadComplete**	All events are finished. Good place for work that needs every control ready.
8	**PreRender**	**Last chance** to change anything on the page before the HTML is made.
9	**SaveStateComplete**	ViewState is saved into the hidden `__VIEWSTATE` field. Changes made after this will not be saved.
10	**Render**	This is a method, not an event. Every control writes its own HTML.
11	**Unload**	Cleanup work. The page cannot be changed now.

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)          // runs only the first time, not on postback
    {
        GridView1.DataSource = GetStudents();
        GridView1.DataBind();
    }
}
```

Important point: on a **postback** the same cycle runs again. The old values come back **between Init and Load**, and the user's event (like Click) runs **after Load**. This is why we write `if (!IsPostBack)` — otherwise the data binding will erase what the user typed.

Q2 (b). Define a connection string in a .NET application. Explain the important parameters commonly included in a SQL Server connection string and state their purposes.

8 Marks

> **Definition.** A **connection string** is one line of text made of `key = value` **pairs separated by semicolons**. It gives the application all the information needed to **find the database and open a connection** — which server, which database, which username and password, and how the connection should behave.

We keep the connection string **outside the code** — in `web.config` for old ASP.NET, or in `appsettings.json` for ASP.NET Core. Then the same program can use a different database in the college lab and in the real server, without compiling again.

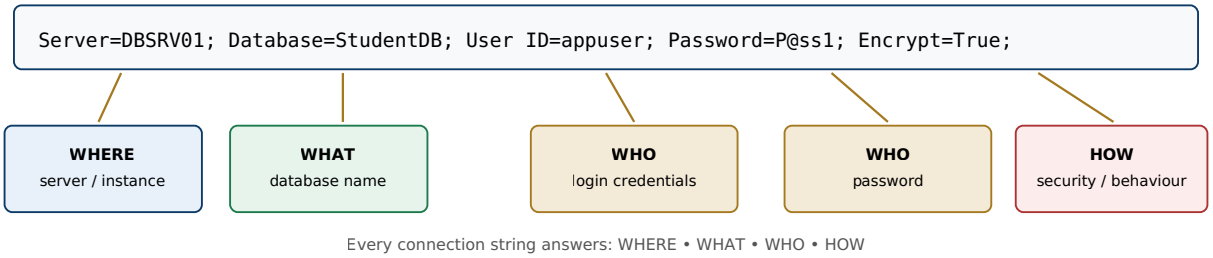

Fig 2.2: Anatomy of a SQL Server connection string

```
// appsettings.json
"ConnectionStrings": {
  "Default": "Server=DBSRV01;Database=StudentDB;User ID=appuser;Password=P@ss1;"
}
```

Important Parameters

Parameter	Purpose	Example
Server (Data Source)	Name or IP address of the computer where SQL Server is running.	`Server=DBSRV01`
Database (Initial Catalog)	Which database to open on that server.	`Database=StudentDB`
Integrated Security	If `True`, the current **Windows account** is used and no password is needed. If `False`, SQL username and password are used.	`Integrated Security=SSPI`
User ID	The SQL Server login name.	`User ID=appuser`
Password	Password of that login. It must be kept safe, never written inside the code.	`Password=P@ss1`
Connect Timeout	How many seconds to wait before giving an error if the server does not reply. Default is 15.	`Connect Timeout=30`
Encrypt	Whether the data travelling between application and server is encrypted.	`Encrypt=True`
TrustServerCertificate	If `True`, the server's certificate is not checked. Use only while developing.	`TrustServerCertificate=False`
Pooling / Max Pool Size	Turns on **connection pooling**. Old connections are reused instead of making new ones, so the application becomes faster.	`Pooling=True`
MultipleActiveResultSets	Allows more than one query to be open on the same connection at the same time.	`MARS=True`
Application Name	Shows your application's name in SQL Server monitoring tools.	`Application Name=StudentApp`
AttachDbFilename	Attaches an `.mdf` database file directly. Used with LocalDB during practice.	`AttachDbFilename=...\db.mdf`

Best Practices

- Keep the connection string in a **configuration file** or environment variable, never inside the C# code.
- Use **Windows authentication** if possible, so no password is written anywhere.
- Always close the connection using a `using` block, so it goes back to the pool quickly.
- Never show the connection string to the user or upload it to GitHub.

Q3 (a). Explain the role and structure of the appsettings.json file in ASP.NET Core and discuss its importance in configuration management.

7 Marks

> **Role.** `appsettings.json` is the **main settings file of an ASP.NET Core application**. It takes the place of the old `web.config` file. It keeps things like **connection strings, logging levels and application settings** in simple JSON form. When the application starts, this file is loaded automatically into `IConfiguration`, and any class can read it.

Structure

The file is written in **sections**. To read a value we use a path with colons, like `"AppSettings:AppName"`.

```
{
  "Logging": {
    "LogLevel": { "Default": "Information" }
  },
  "ConnectionStrings": {
    "Default": "Server=.;Database=StudentDB;Trusted_Connection=True;"
  },
  "AllowedHosts": "*",
  "AppSettings": {                    // our own section
    "AppName": "Student Portal",
    "MaxPageSize": 50
  }
}
```

- **Logging** — how much detail should be written in the log.
- **ConnectionStrings** — database connection details.
- **AllowedHosts** — which host names are allowed, for security.
- **Custom section** — any setting we want to add ourselves.

How to Read the Values

```
// simple way
string name = config["AppSettings:AppName"];
string cs   = config.GetConnectionString("Default");

// better way — Options pattern (binds the section to a class)
builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));
```

Layers of Configuration

`appsettings.json` is only one source of settings. Many sources are loaded one after another, and the **later one wins** if the same key is repeated.

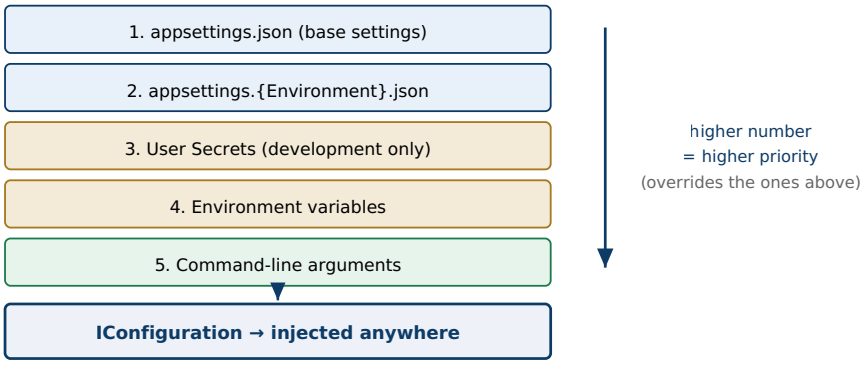

Fig 3.1: Layered configuration providers merged into one IConfiguration

Importance in Configuration Management

1. **Different settings for different environments:** files like `appsettings.Development.json` and `appsettings.Production.json` are loaded on top of the main file. So the lab machine and the real server can use different databases **without changing the code**.
2. **Settings are separate from code:** changing a server name or a log level is just editing a text file. No need to compile and deploy again.
3. **Security:** because environment variables and user secrets come **after** the JSON file, the real password can be kept outside the project and never uploaded to GitHub.
4. **Strong typing:** using the **Options pattern**, a section is bound to a normal C# class. So we get IntelliSense and spelling mistakes are caught early.
5. **Automatic reload:** if a value is edited while the application is running, ASP.NET Core can pick up the new value **without restarting** the application.
6. **One central place:** all settings stay in one file and reach any class through dependency injection.

> **Conclusion:** `appsettings.json` makes the application **configurable instead of hard-coded**. The same compiled application can run in development, testing and production just by changing the settings.

Q3 (b). Discuss the role and major advantages of Entity Framework Core in performing database access operations.

8 Marks

> **Definition. Entity Framework Core (EF Core)** is Microsoft's **ORM (Object Relational Mapper)** for .NET. A database stores data in **tables**, but C# works with **objects**. EF Core sits between them: we write normal C# classes and LINQ queries, and EF Core makes the SQL, runs it, and converts the returned rows back into objects.

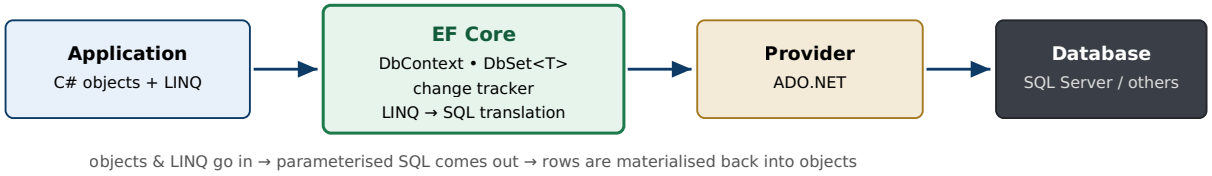

Fig 3.2: EF Core sits between application objects and the relational database

Role of EF Core (what it does)

- **Mapping:** it connects a class to a table and a property to a column. It also handles relationships like one-to-many.
- **Query translation:** it converts our LINQ query into **SQL**, sends it to the database, and turns the result into objects.
- **Change tracking:** the `DbContext` remembers which objects were added, changed or deleted. When we call `SaveChanges()`, all of them are saved together in **one transaction**.
- **Migrations:** commands like `Add-Migration` and `Update-Database` create and update the database tables from our classes, so the database always matches the code.

Small Example

```
public class Student                        // this class becomes a table
{
    public int Id { get; set; }
    public string Name { get; set; }
    public double GPA { get; set; }
}

public class SchoolContext : DbContext
{
    public DbSet<Student> Students { get; set; }   // the "Students" table
}

// using it
db.Students.Add(new Student { Name = "Arpan", GPA = 3.9 });   // insert
var list = db.Students.Where(s => s.GPA >= 3.5).ToList();     // select
db.SaveChanges();                                             // saves everything
```

Major Advantages

1. **Less code:** we do not have to write connection, command and reader code, and no SQL for normal work. Insert, update and delete become one or two lines of C#.
2. **Errors caught early:** LINQ queries are normal C# code, so a wrong column name or wrong type is shown by the compiler. In plain SQL strings such mistakes are found only when the program runs.
3. **Works with many databases:** SQL Server, MySQL, PostgreSQL, SQLite, Oracle and others. Changing the database mostly means changing the provider and the connection string.

4. **Safe from SQL injection:** EF Core always sends values as **parameters**, so a hacker cannot inject SQL through a text box.
5. **Migrations:** the database structure can be changed step by step and kept together with the code, so all team members have the same tables.
6. **Good performance options:** `AsNoTracking()` for read-only queries, control over how related data is loaded, and full **async** support (`ToListAsync`) for websites with many users.
7. **Easy to test:** the InMemory provider lets us test the code without a real database.
8. **Free and cross-platform:** it is open source and runs on Windows, Linux and Mac.

> **One limitation (write this for balance):** for very complex or very fast queries, the SQL made by EF Core may not be the best. In that case we can still write our own SQL using `FromSqlInterpolated()` or call a stored procedure. Both ways can be used in the same project.

Q4 (a). Explain the exception-handling mechanism in C#. Discuss the functions of the try, catch and finally blocks with an appropriate example.

7 Marks

> **Definition.** An **exception** is an error that happens while the program is running — like dividing by zero, using a wrong array index, or opening a file that does not exist. In C#, an exception is an object of the `System.Exception` class. **Exception handling** means catching such errors using **try, catch and finally** blocks so that the program does not crash and can continue in a proper way.

Function of Each Block

- `try` — we put the risky code inside it. This is the code that may cause an error.
- `catch` — it **handles** one type of error. We can write many catch blocks. They are checked from top to bottom and only the **first matching one** runs.
- `finally` — this block **always runs**, whether an error happened or not. We use it for cleaning work like closing a file or a database connection.
- `throw` — used to raise an error ourselves. Writing only `throw;` inside a catch sends the same error upwards without losing the error details.

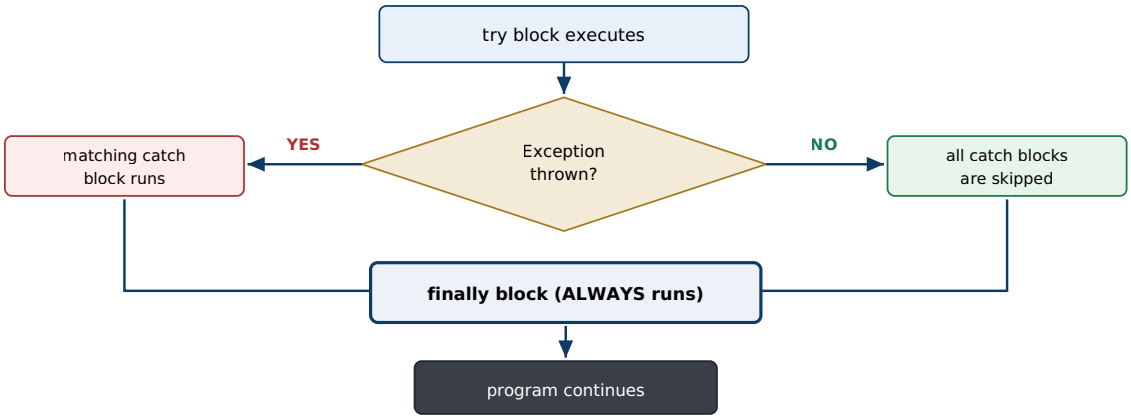

Fig 4.1: Control flow of try–catch–finally

Example

```
try
{
    int[] marks = { 90, 85, 78 };
    int idx = int.Parse(Console.ReadLine());
    int div = int.Parse(Console.ReadLine());
    Console.WriteLine(marks[idx] / div);       // error may happen here
}
catch (DivideByZeroException ex)               // most specific error first
{
    Console.WriteLine("Cannot divide by zero.");
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine("Index is not correct.");
}
catch (Exception ex)                           // general error LAST
{
    Console.WriteLine("Some other error: " + ex.Message);
}
finally
{
    Console.WriteLine("Closing resources...");  // this always runs
}
```

Important Rules

1. Catch blocks must be written from **specific to general**. If we write `catch (Exception)` first, it will catch everything and the compiler gives an **error**.
2. `finally` runs even if there is a `return` inside try or catch. This is how it makes sure cleaning always happens. The `using` statement is just a short form of try–finally.
3. If **no catch matches**, the error goes to the calling method. If nobody handles it, the program stops.
4. Useful properties of an exception object: `Message` (what went wrong), `StackTrace` (where it went wrong), `InnerException` and `Source`.
5. We can make our **own exception** by inheriting the `Exception` class, for example `class InvalidGpaException : Exception`.
6. Do not use exceptions for normal checking. Exceptions are slow, so use simple validation or `TryParse` where possible.

> **Benefits:** the program does not crash, error code stays separate from normal code, resources are always closed, we get full error details for finding the problem, and the user sees a proper message instead of a crash screen.

Q4 (b). Define Language Integrated Query (LINQ). Explain how LINQ enables querying and manipulation of data from different data sources within .NET applications. 8 Marks

> **Definition. LINQ (Language Integrated Query)** was introduced in C# 3.0. It brings **query writing directly inside the C# language**. Using one SQL-like syntax we can search and work with **many kinds of data** — lists and arrays in memory, database tables, XML files and DataSets. So we do not need to learn a separate query language for each one.

One Syntax, Many Data Sources

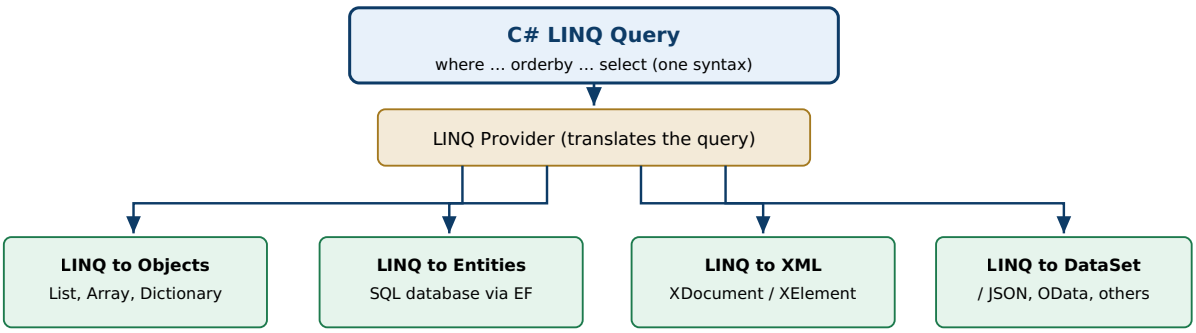

Fig 4.2: The same LINQ query works over any data source through its provider

The part that does the actual work is called the **provider**. Our query stays the same, but the provider changes underneath — for a list it simply loops, and for a database it makes SQL.

Two Ways of Writing a LINQ Query

```
// (a) Query syntax – looks like SQL
var q1 = from s in students
         where s.GPA >= 3.5
         orderby s.GPA descending
         select s.Name;

// (b) Method syntax – uses lambda. Both give the same result.
var q2 = students.Where(s => s.GPA >= 3.5)
                 .OrderByDescending(s => s.GPA)
                 .Select(s => s.Name);
```

Common LINQ Operators

Work	Operators	Meaning
Filtering	`Where`	Take only the items that match a condition
Selecting	`Select`	Choose which fields you want in the result
Sorting	`OrderBy`, `ThenBy`	Arrange the result in order
Grouping / joining	`GroupBy`, `Join`	Group similar items, or join two sources
Calculating	`Count`, `Sum`, `Min`, `Max`, `Average`	Get one single value from many items
Single item	`First`, `Single`, `Any`, `All`	Take one item or check a condition
Converting	`ToList`, `ToArray`	Run the query and store the result

How LINQ Works with Different Data Sources

1. **Same syntax everywhere:** the same `Where` and `OrderBy` work on a `List`, on a database table and on an XML file. Only the provider changes.
2. **Query becomes SQL for databases:** when we query a database through EF Core, LINQ is converted into **SQL and run on the server**. So only the needed rows come over the network, and the values are sent as parameters, which stops **SQL injection**.
3. **Mistakes caught by the compiler:** a LINQ query is normal C# code, so a wrong column name or wrong type gives a **compile-time error**. If we write SQL as a string, the mistake is found only after running the program.
4. **Result is typed:** we get proper objects, not rows that must be converted with casting. So there are fewer run-time errors.
5. **Short and easy to read:** we write **what** we want, not **how** to loop. Many lines of `foreach` and `if` become one clear query.
6. **Deferred execution:** the query does not run when we write it. It runs only when we use the result (in a `foreach` or with `ToList()`). So we can build a query in parts — for example add paging or a filter only if the user selected it — and it always uses the latest data.
7. **Queries can be joined together:** filtering, sorting, grouping and calculating can be chained one after another. Using `AsParallel()`, a large in-memory query can even run on many CPU cores.

> **Conclusion:** LINQ turns data searching from long loops and risky SQL strings into **short, safe, easy-to-read C# code**, and it gives one common way to work with every data source in .NET.

Q5 (a). Discuss asynchronous programming in .NET with reference to the async and await keywords and explain its benefits. 7 Marks

> **Definition. Asynchronous programming** means the program can **start a slow job** — like reading a file, calling a web service or running a database query — and **do other work while waiting**, instead of standing idle. In .NET this is done with a `Task` (which means "a job that will finish later") and the two keywords `async` and `await`.

The Two Keywords

- `async` — written before a method. It allows us to use `await` inside that method. An async method returns `Task` or `Task<T>` (and `void` only for event handlers).
- `await` — written before a Task. It means "wait for this job, but **do not block**". The method stops there, gives the thread back so it can do other work, and continues automatically from the same line when the job is finished.

Very important point: `await` does **not** create a new thread. It **frees** the current thread while the operating system does the slow work.

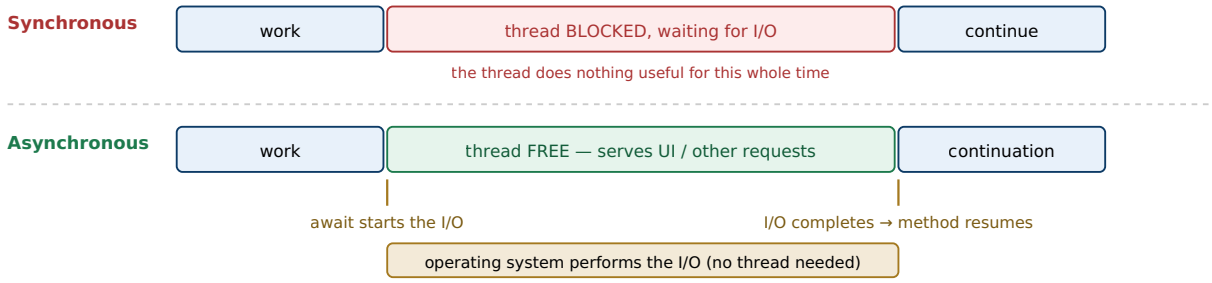

Fig 5.1: await frees the thread during I/O instead of blocking it

Example

```
// BAD — normal (blocking) way: the thread just waits and does nothing
string html = http.GetStringAsync(url).Result;      // .Result blocks!

// GOOD — asynchronous way
public async Task<int> GetPageLengthAsync(string url)
{
    string html = await http.GetStringAsync(url);   // thread is free here
    return html.Length;                             // continues after download
}

// in a Web API
public async Task<IEnumerable<Student>> Get() => await _db.Students.ToListAsync();
```

Benefits

1. **The screen does not freeze:** in a desktop or mobile app, the UI thread becomes free at every `await`. So the window keeps moving and the user can still click. Without this, the app shows "Not Responding".
2. **The server can handle more users:** in ASP.NET Core, while one request is waiting for the database, its thread goes back to the pool and serves another user. So the **same server handles many more users** without buying new hardware. This is the biggest benefit.
3. **Saves memory:** every thread takes about 1 MB of memory. Async work does not keep threads waiting, so less memory and less switching between threads.

4. **Many jobs at the same time:** we can start three downloads together and wait for all with `await Task.WhenAll(t1, t2, t3)`. Then the total time is the time of the slowest one, not the sum of all.
5. **Code stays easy to read:** the code still looks like normal step-by-step code and works with `try/catch` and loops. The old callback style was very hard to read.
6. **Can be cancelled:** using a `CancellationToken`, a long job can be stopped cleanly if the user presses Cancel.

> **Rules to remember:** use **async all the way** — never use `.Result` or `.Wait()`, because they block the thread and can cause a **deadlock**. Give async methods a name ending with **Async**. Use `Task.Run` only for heavy calculation work, not for file or network work.

Q5 (b). Explain the SOLID principles and discuss their importance in developing maintainable and extensible software applications.

8 Marks

SOLID is a short form of five rules of object-oriented design given by Robert C. Martin. If we follow these five rules, our software becomes **easy to understand, easy to change, easy to test and easy to extend**. ASP.NET Core itself is built by following these rules.

Fig 5.2: The five SOLID principles

The Five Principles

Principle	What it says (in simple words)	Example
S — Single Responsibility	One class should do **only one job**. It should have only one reason to change.	Do not make one class that calculates the result, prints it and saves it. Make three classes, one for each job.
O — Open / Closed	A class should be **open for adding new things but closed for changing old things**.	To add a new payment type, write a new class that implements `IPayment`. Do not edit the old working code.
L — Liskov Substitution	A **child class must work correctly wherever the parent class is used**, without breaking anything.	`FileStream` and `MemoryStream` both work wherever `Stream` is expected. But a `Square` class inherited from `Rectangle` breaks this rule.
I — Interface Segregation	Do not force a class to implement methods it does **not need**. Make many small interfaces instead of one big one.	Make `IReadRepo` and `IWriteRepo` instead of one big `IRepository` with 20 methods.
D — Dependency Inversion	A class should depend on an **interface**, not on another concrete class.	A controller uses `IMessageService` given by the DI container, instead of writing `new EmailService()` itself.

Small Example (SRP and DIP together)

```
// BAD: this class saves, sends email and writes log — three jobs, and it
// creates its own objects, so we cannot change or test it easily.

// GOOD: one job, and the helpers come from outside through interfaces
public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly INotifier _notifier;

    public OrderService(IOrderRepository repo, INotifier notifier)   // DIP
        { _repo = repo; _notifier = notifier; }

    public void Place(Order o) { _repo.Save(o); _notifier.Notify(o); }
}
// To send SMS instead of email, we only register a different INotifier.
```

Importance in Maintainable and Extensible Software

1. **Easy to maintain:** because each class does one job, a change stays in one place. Fixing one thing does not break another thing.
2. **Easy to add new features:** a new feature comes as a **new class**, so the old tested code is not touched. This means fewer new bugs.
3. **Programs behave correctly:** Liskov's rule makes sure that child classes do not surprise us when used in place of the parent.
4. **Loose coupling:** because classes depend on interfaces, we can change the database, the logger or the payment gateway without changing the whole application.
5. **Easy to test:** interfaces can be replaced by **fake (mock) objects** in unit tests, so tests run fast without a real database or internet.
6. **Easy for the team:** small classes are easy to read and understand, so new members learn the project faster and many people can work at the same time.
7. **Reusable code:** small and independent classes can be used again in other projects.
8. **Base of good design:** patterns like Repository, dependency injection, middleware and microservices are all built on SOLID.

Conclusion: SOLID removes **rigidity** (hard to change), **fragility** (breaks somewhere else) and **immobility** (cannot reuse) from our code. The result is software that is cheaper to change, safer to extend and easier to test.

OR — Alternative for Q5 (b)

Q5 (b) OR. Explain the concept of state management in ASP.NET applications. Discuss the different client-side and server-side state management techniques with suitable examples.

8 Marks

> **Why it is needed. HTTP is a stateless protocol**. This means the server treats every request as new and **forgets everything** after sending the reply — the values in the text boxes, the login details, the shopping cart, everything. **State management** is the set of ways ASP.NET gives us to **save this data** between requests and postbacks.

These ways are of two types: **client-side** (data is kept in the browser) and **server-side** (data is kept on the server).

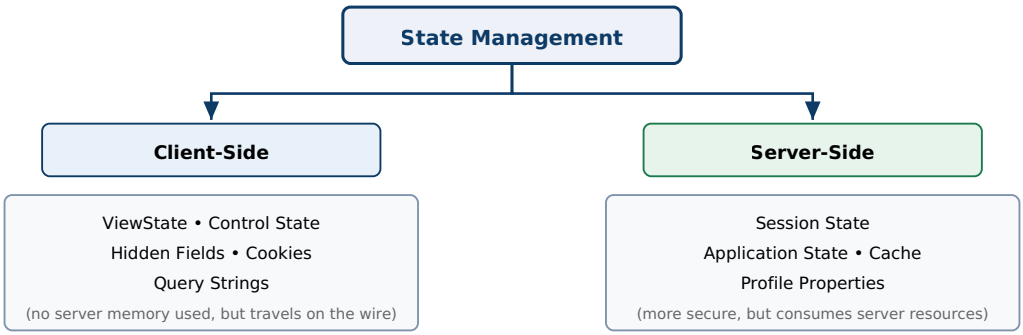

Fig 5.3: Classification of ASP.NET state management techniques

A. Client-Side Techniques

1. **ViewState** — ASP.NET automatically saves the values of the page controls in a hidden field called `__VIEWSTATE` and brings them back on the next postback of the **same page**. It does not use server memory, but it makes the page **heavier** because the data travels every time.
   Example: `ViewState["Count"] = 5;`
2. **Control State** — a small part of ViewState that **cannot be turned off**. It is used for data that a control needs to work properly.
3. **Hidden Fields** — store a small value inside the form. Anyone can see it with "View Source", so **never** keep secret data here.
4. **Cookies** — small text files (about 4 KB) saved in the browser and sent with every request. They can be temporary (deleted when browser closes) or permanent (with an expiry date). Good for settings like theme and "remember me". The user can delete or edit them, so no secret data.
   Example: `Response.Cookies["theme"].Value = "dark";`
5. **Query String** — data added at the end of the URL, like `Details.aspx?id=5`, and read with `Request.QueryString["id"]`. It is simple and the link can be bookmarked, but it is **visible, short and easy to change** by the user.

B. Server-Side Techniques

1. **Session State** — data of **one user**, saved on the server. The server knows which data belongs to which user through a session ID cookie. It works on all pages until it times out (20 minutes by default). Best for login details and shopping cart, but it uses server memory.
   Example: `Session["Cart"] = cart;`

Modes: *InProc* (fastest, in server memory, lost on restart), *StateServer* (a separate service), *SQLServer* (saved in database, safe for many servers), and *Redis* in ASP.NET Core.

2. **Application State** — data shared by **all users**, like a visitor counter: `Application["Visitors"]`. Since many users may write at the same time, we must use `Application.Lock()` and `UnLock()`. It is lost when the application restarts.
3. **Cache** — data shared by all users with an **expiry time**. It is used to store the result of a heavy database query so that the query is not run again and again. ASP.NET Core gives `IMemoryCache` and `IDistributedCache`.
4. **Profile Properties** — per-user data saved **permanently in a database**. Unlike Session, it stays even after the browser is closed or the server is restarted.

Comparison

Technique	Who can use it	Stored where	How long it stays	Best for
ViewState	One page	Browser (hidden field)	Until you leave the page	Control values of that page
Hidden field	One page	Browser (form)	Until you leave the page	Small non-secret values
Cookies	Whole site	Browser	Until expiry date	Theme, "remember me"
Query string	Between pages	URL	One request	Small values, shareable links
Session	One user	Server	Until timeout	Login data, shopping cart
Application	All users	Server	Until app restarts	Visitor counter, common settings
Cache	All users	Server	Until it expires	Heavy query results
Profile	One user	Database	Permanent	Long-term user settings

> **How to choose:** use **client-side** for small and non-secret data (server memory is saved). Use **server-side** for secret or large data (safer, but uses server memory).
> **Note:** in ASP.NET Core there is no ViewState and no Application state. Their work is done by **TempData, Session and caching services**.

Q6 (a). Explain dependency injection in ASP.NET Core. Discuss its advantages and describe the commonly used service lifetimes. 7 Marks

Definition. Normally a class creates the objects it needs by writing `new`. In **Dependency Injection (DI)**, the class does **not** create them. It only says which **interface** it needs, and a **container** gives (injects) the ready object from outside. ASP.NET Core has this container built in: we register the services in `Program.cs`, and the container supplies them automatically, usually through the **constructor**.

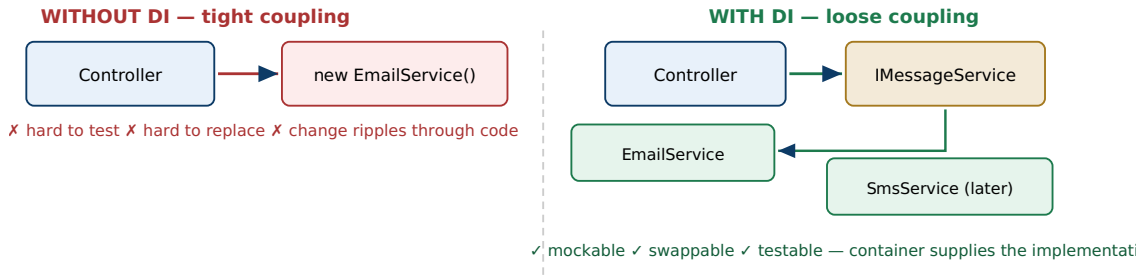

Fig 6.1: DI decouples the consumer from the concrete implementation

How to Use It (four steps)

```
// 1. Make an interface
public interface IMessageService { void Send(string to, string msg); }

// 2. Make a class that implements it
public class EmailService : IMessageService { ... }

// 3. Register it in Program.cs
builder.Services.AddScoped<IMessageService, EmailService>();

// 4. Ask for it in the constructor — the container gives it automatically
public class NotifyController : Controller
{
    private readonly IMessageService _svc;
    public NotifyController(IMessageService svc) { _svc = svc; }
}
```

Later, if we want to send SMS instead of email, we change **only the registration line**. The controller code is not touched at all.

Service Lifetimes

Lifetime means how long one object lives before a new one is made.

Lifetime	How to register	When a new object is made	Used for
Transient	`AddTransient<I,T>()`	**Every time** it is asked for — a new object each time.	Small, light services that keep no data
Scoped	`AddScoped<I,T>()`	**One object per request** — the same object is used everywhere inside that one request.	`DbContext`, repositories
Singleton	`AddSingleton<I,T>()`	**Only one object for the whole application** — shared by all users.	Cache, configuration, logger

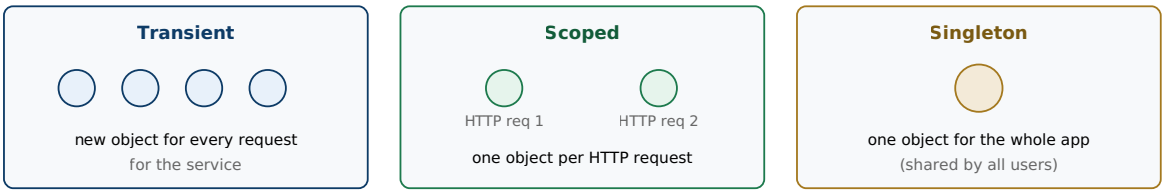

Fig 6.2: The three service lifetimes

> **Important rule:** never put a **Scoped** service inside a **Singleton**. The singleton will keep that object forever, so a `DbContext` would be shared by all users, which is not safe. ASP.NET Core shows this error when the application starts.

Advantages of Dependency Injection

1. **Loose coupling:** a class depends on an **interface**, not on another class. So one part can be changed without disturbing others.
2. **Easy testing:** in unit tests we can give a **fake object** instead of the real one, so tests do not need a real database or email server.
3. **Easy to change:** to use a different implementation, only the registration line changes — not the whole project.
4. **Object life is managed automatically:** the container creates the objects and also **disposes** them at the right time.
5. **Cleaner code:** no `new` written everywhere, and common services like logging and configuration are shared properly.
6. **Already used by the framework:** ASP.NET Core itself gives `ILogger`, `IConfiguration` and `DbContext` through the same container.

Q6 (b). Explain the development of RESTful Web APIs using ASP.NET Core. Discuss HTTP methods, endpoints, request handling, response handling and status codes. 8 Marks

Definition. A **Web API** is a service that gives data over HTTP, so that many kinds of clients — browsers, mobile apps, desktop apps and other servers — can use it. A **RESTful** Web API follows the REST style: each thing (like a student) is called a **resource** and has its own **URL**; the standard **HTTP methods** are used to work on it; each request is **independent (stateless)**; and the data is usually sent as **JSON**. In ASP.NET Core we make it with a controller that inherits `ControllerBase`.

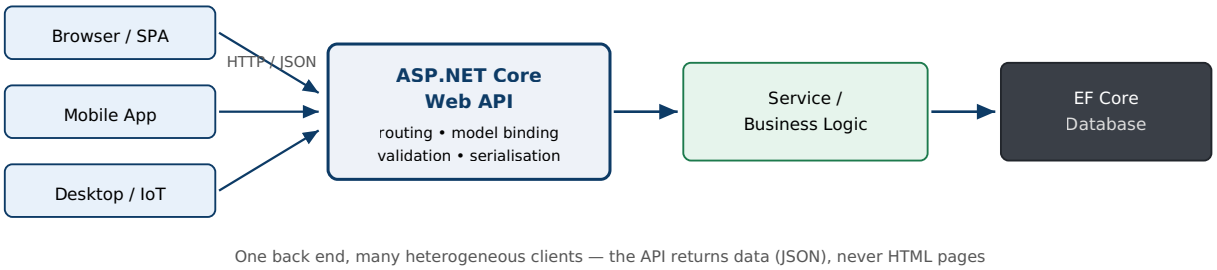

Fig 6.3: Multiple clients consuming a single ASP.NET Core Web API

1. Creating the API and its Endpoints

An **endpoint** means **one URL plus one HTTP method**, which together do one job. ASP.NET Core finds the correct method using **routing attributes**.

```
[ApiController]
[Route("api/[controller]")]              // URL becomes api/students
public class StudentsController : ControllerBase
{
    [HttpGet]                            // GET api/students → give all
    public async Task<IEnumerable<Student>> GetAll() => await _db.Students.ToListAsync();

    [HttpGet("{id}")]                    // GET api/students/5 → give one
    public async Task<IActionResult> Get(int id)
    {
        var s = await _db.Students.FindAsync(id);
        return s is null ? NotFound() : Ok(s);       // 404 or 200
    }

    [HttpPost]                           // POST api/students → add new
    public async Task<IActionResult> Create(Student s)
    {
        _db.Students.Add(s);
        await _db.SaveChangesAsync();
        return CreatedAtAction(nameof(Get), new { id = s.Id }, s);   // 201
    }
}
```

The `[ApiController]` attribute automatically checks the data sent by the user and returns **400 Bad Request** if it is wrong, so we do not have to write that check ourselves.

2. HTTP Methods and Endpoints

Method	Example endpoint	What it does	Success code
GET	`/api/students` or `/api/students/5`	Read all records, or read one record	200 OK
POST	`/api/students`	Add a new record	201 Created
PUT	`/api/students/5`	Update the full record	204 No Content
PATCH	`/api/students/5`	Update only some fields	200 / 204
DELETE	`/api/students/5`	Delete the record	204 No Content

Rules for making good endpoints: use **nouns in plural** (`/api/students`) and never verbs — `/api/getStudent` is wrong because GET is already the verb. Use nesting for related data (`/api/students/5/courses`), use the query string for filtering and paging (`?page=2`), and add a version like `/api/v1/students`.

3. Request Handling (what happens when a request comes)

- **Routing** — matches the URL and method with the correct action method.
- **Model binding** — automatically fills the method parameters from the request: `[FromRoute]` (from URL), `[FromQuery]` (from query string), `[FromBody]` (from JSON body).
- **Validation** — checks the rules written on the model like `[Required]` and `[Range]`. If wrong, **400** is returned automatically.
- **Middleware** — before the controller runs, middleware does common work like error handling, HTTPS, CORS, authentication and authorization.

4. Response Handling (what we send back)

- We use helper methods to return the result: `Ok(data)`, `CreatedAtAction(...)`, `NoContent()`, `NotFound()`, `BadRequest()`, `Unauthorized()`.
- The object is **automatically converted into JSON**, and the correct content type header is set.
- We should return a **DTO** (a small class with only the needed fields) instead of the database entity, so that password and internal fields are not shown.
- A global error-handling middleware turns any unhandled error into a clean **500** reply, instead of showing the error details to the user.
- **Swagger** is used to see and test all endpoints from the browser.

5. HTTP Status Codes

Group	Code	Meaning
2xx — success	200 OK	Everything is fine and data is returned
	201 Created	New record was created
	204 No Content	Work is done but there is nothing to send back
4xx — user's mistake	400 Bad Request	The data sent is wrong or incomplete
	401 Unauthorized	Not logged in — **authentication** failed
	403 Forbidden	Logged in but not allowed — **authorization** failed
	404 Not Found	That record does not exist
	409 Conflict	Two users changed the same record
5xx — server's mistake	500 Internal Server Error	An error happened on the server
	503 Service Unavailable	Server is down or too busy

Advantages of Web API

1. **Any client can use it** — one back end serves website, Android, iOS and IoT devices.
2. **Light and fast** — it uses simple JSON instead of heavy XML like SOAP, which is good for mobile data.
3. **Uses all HTTP features** — caching, status codes, headers and versioning.
4. **Stateless, so easy to scale** — requests can be shared between many servers.
5. **Many features already built in** — routing, validation, dependency injection and security (JWT).
6. **Easy to test** — endpoints can be tested with Postman or Swagger without making any UI.

OR — Alternative for Q6 (b)

Q6 (b) OR. Discuss the benefits and challenges of adopting a microservices approach in enterprise .NET applications. 8 Marks

> **Definition.** In the **microservices** approach, one big application is divided into many **small and independent services**. Each service does **one business work** (Orders, Payments, Inventory), runs as its **own program**, has its **own database**, and talks to other services over HTTP, gRPC or a message queue. This is the opposite of a **monolith**, where everything is built and deployed as one single unit.

In .NET, each microservice is usually an ASP.NET Core Web API, packed in a **Docker** container and managed by **Kubernetes**.

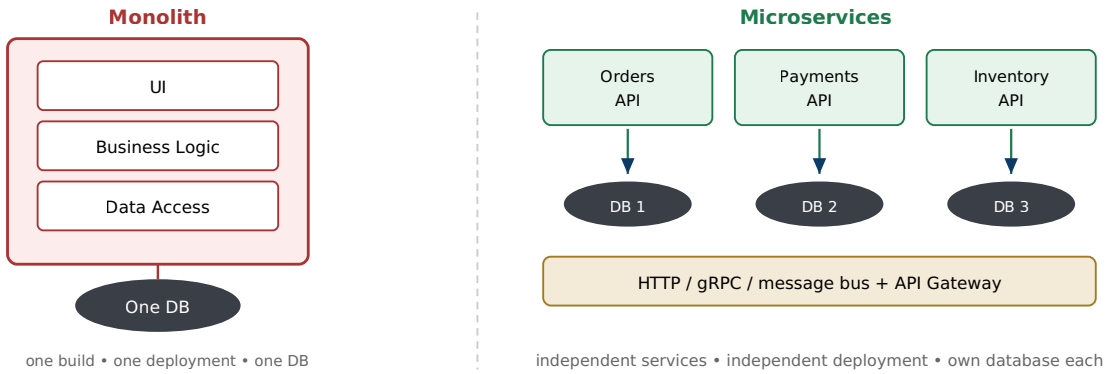

Fig 6.4: Monolith versus microservices

Benefits

1. **Deploy separately:** each service can be updated and released alone. So a small change does not need the whole system to be deployed again, and the risk is much less.
2. **Scale only what is needed:** if only the Orders service is busy during a sale, we run more copies of **only that service**. In a monolith we would have to copy the whole application, which costs much more.
3. **One failure does not stop everything:** if the Payment service fails, the rest of the application still works — if we use patterns like **retry, timeout and circuit breaker** (the Polly library in .NET).
4. **Freedom to choose technology:** each team can use a different .NET version, a different language, or a different database that suits its service.
5. **Teams work independently:** a small team owns one service completely. So teams do not have to wait for each other, which is very useful in big companies.
6. **Easy to understand and replace:** a small service can be learned quickly and even rewritten completely, which is impossible in a very large monolith.

Challenges

1. **The system becomes distributed:** a simple method call now becomes a **network call**. So we must handle network delay, failures, retries and timeouts everywhere. Finding a bug across many services is difficult.

2. **Data consistency problem:** each service has its own database, so we **cannot use one transaction** across services. We must use **eventual consistency** and the **Saga pattern**, which is much harder than one simple `SaveChanges()`.
3. **More work for operations (DevOps):** many services need containers, Kubernetes, service discovery, an **API gateway** and secret management. A strong DevOps team is required.
4. **Monitoring is difficult:** we need central logging, **distributed tracing** and health checks just to know what is happening inside the system.
5. **Testing is harder:** testing the whole flow across many separately deployed services is much more complex than testing one program.
6. **More security work:** every call between services must be secured, and this also adds extra network cost.
7. **Costly for small projects:** it needs skilled people and extra effort. For a small application the trouble is more than the benefit.

> **Conclusion:** microservices give **independent deployment, better scaling and better fault isolation**, but they bring **network complexity, data consistency problems and heavy DevOps work**. So the common advice is: **start with a well-organised monolith**, and move to microservices only when the application and the team really become large.

Q7 (a). Short Note: Garbage Collection mechanism in .NET

5 Marks

> **Definition. Garbage Collection (GC)** is the automatic memory cleaning system of the CLR. Objects are created on the **managed heap**. When an object is **no longer used by any variable**, the GC finds it and frees that memory automatically. So in C# we never write `free()` or `delete`, and problems like memory leaks and dangling pointers do not happen.

Steps of a Collection

1. **Marking:** the GC starts from the **roots** (static variables, local variables of running methods) and follows every reference. Whatever it can reach is **marked as alive**. Whatever it cannot reach is garbage.
2. **Relocating:** the references of the surviving objects are updated to their new addresses.
3. **Sweeping and compacting:** the memory of dead objects is freed, and the living objects are **moved close together**. So the free memory becomes one big block. This removes gaps and makes the next object creation very fast.

Generations

The GC works with **generations**, based on one simple idea: **most objects die very quickly**. So instead of checking the whole memory every time, it checks only the new objects. This keeps the pause very short.

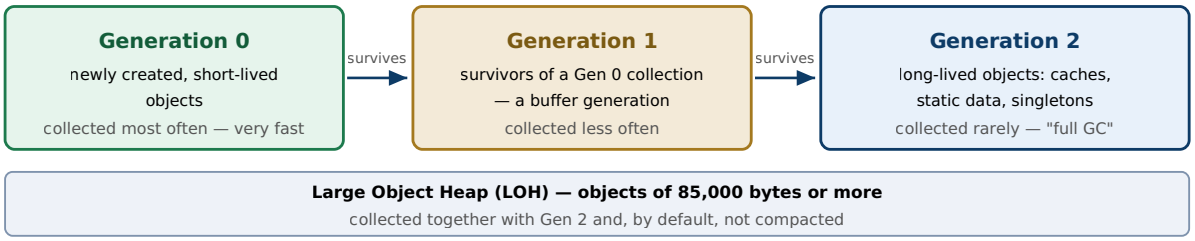

Fig 7.1: The generational managed heap in .NET

- **Gen 0** — new objects. Cleaned most often and very fast. Most objects die here.
- **Gen 1** — objects that survived Gen 0. It works like a middle stage.
- **Gen 2** — old objects that live long, like cache and static data. Cleaning this is the slowest, so it is done rarely.
- **Large Object Heap (LOH)** — objects of 85,000 bytes or more (big arrays). Cleaned together with Gen 2 and normally not compacted.

When Does the GC Run?

- When Gen 0 memory becomes full (this is the most common reason).
- When the computer is running low on memory.
- When `GC.Collect()` is called by the programmer — but this is **not recommended** in normal code.

Types and Related Points

- **Workstation GC and Server GC:** Workstation GC is for desktop applications and keeps the app responsive. **Server GC** uses one heap for each CPU core for high speed, and it is the default in ASP.NET Core.

- **Background GC:** the slow Gen 2 cleaning runs in the background while the application keeps working, so the pause is small.
- **Dispose and IDisposable:** the GC frees only **managed memory**. Things like file handles and database connections are **unmanaged**, so we must close them ourselves using `Dispose()` or a `using` block. A finalizer (`~ClassName`) is only a backup and it actually **delays** the cleaning.

Advantages

1. Memory errors like leaks and dangling pointers are removed, so the program is more reliable.
2. Creating an object is very fast, and memory gaps are removed automatically.
3. Because of generations and background cleaning, the program stops for only a very short time.
4. The programmer can think about the actual logic instead of memory management.

Q7 (b). Short Note: Authentication and Authorization — 5 Marks

Authentication answers **"Who are you?"** — it checks the identity of the user with a username, password, token or certificate.
Authorization answers **"What are you allowed to do?"** — it checks whether that known user has permission for this work.
Authentication always comes first. That is why in ASP.NET Core we write `UseAuthentication()` before `UseAuthorization()`.

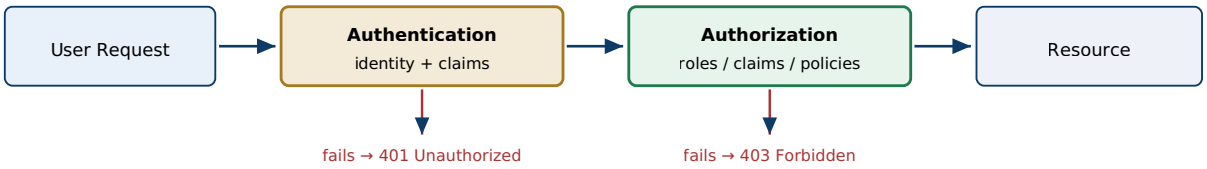

Fig 7.2: Security pipeline — authenticate first, then authorize

Authentication Methods

1. **Forms Authentication** (old ASP.NET) — the user is sent to a login page. After a correct login, an **encrypted ticket** is saved in a cookie and checked on every request.
2. **Windows Authentication** — uses the user's Windows / Active Directory account. Best for office and intranet applications, because no login page is needed.
3. **ASP.NET Core Identity** — a complete ready-made system: user and role tables in the database, password hashing, account lock after wrong attempts, and two-factor authentication.
4. **Cookie Authentication** — after login, a **signed cookie** carries the user's details on every request. This is the normal choice for websites.
5. **JWT Bearer Token** — used for Web APIs and mobile apps. The server gives a signed **token**, and the client sends it in the header of every request. It is **stateless**, so it works well with many servers.
6. **External login (OAuth 2.0 / OpenID Connect)** — login with Google, Facebook or Microsoft.

Authorization Methods

1. **Simple:** `[Authorize]` allows **any logged-in user**. `[AllowAnonymous]` allows everyone.
2. **Role-based:** permission depends on the role — `[Authorize(Roles = "Admin")]`.
3. **Claims-based:** permission depends on a **claim**, which is a small fact about the user, such as Department = IT or Age = 21.
4. **Policy-based:** we make a named rule (policy) that can combine many conditions. This is the modern and recommended way in ASP.NET Core.
5. **Resource-based:** checking on a particular item — for example, only the owner of a document can edit it.

```
app.UseAuthentication();     // who are you?
app.UseAuthorization();      // what can you do?

[Authorize(Roles = "Admin")]
public IActionResult DeleteStudent(int id) { ... }
```

> **Summary:** authentication **checks who the user is** (Forms, Windows, Identity, cookie, JWT), and authorization **decides what that user can do** (roles, claims, policies). If authentication fails we get **401 Unauthorized**, and if authorization fails we get **403 Forbidden**.

Q7 (c). Short Note: Abstract Class vs Interface

5 Marks

Both **abstract class** and **interface** are used for **abstraction and polymorphism** in .NET, and we cannot create an object of either one. The main difference is in their purpose. An abstract class shows an **"is-a" relation with some ready-made code**, while an interface shows only a **"can-do" capability**.

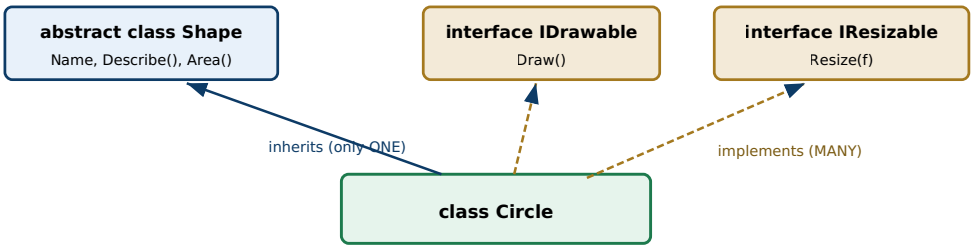

Fig 7.3: One base class may be inherited, but many interfaces may be implemented

Difference

Basis	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Code inside	Can have both empty methods and **full working methods**, plus fields and constructors	Normally only method names; from C# 8 a default body is allowed, but no fields and no constructors
How many	A class can inherit **only one** abstract class	A class can implement **many** interfaces
Members	Fields, properties, methods, events, constructors; any access modifier	Methods, properties, events; all are **public** by default
Can store data?	Yes, it can have variables	No, it cannot store data
Object creation	Not possible directly	Not possible directly
Adding a new method later	Old child classes keep working	All classes that implement it **break**, unless a default body is given
Speed	Slightly faster	Very slightly slower
Meaning	"**is-a**" — a Dog *is an* Animal	"**can-do**" — a Dog *can be* compared

Example

```
public abstract class Shape                 // common data + common code
{
    public string Name { get; set; }        // data is allowed here
    public abstract double Area();          // child must write this
    public void Describe() => Console.WriteLine(Name);   // ready-made code
}

public interface IDrawable { void Draw(); }     // only a capability

public class Circle : Shape, IDrawable          // one class + many interfaces
{
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
    public void Draw() => Console.WriteLine("Drawing circle");
}
```

When to Use Which

Use an Abstract Class when…	Use an Interface when…
Many **related** classes need to share the same code and data (like `Shape` with a `Name`).	**Unrelated** classes need the same ability, like `IComparable` or `IDisposable`.
You need **constructors, fields** or private members in the base class.	One class needs **many** abilities, because C# does not allow more than one base class.
The base class may get new methods later, and old child classes should not break.	You are using **dependency injection and unit testing**, where a fake object replaces the real one.
You want to give a default behaviour that most child classes can use as it is.	You are making a **contract** for other people, and they should not get any code from you.

Conclusion: use an **abstract class to share code** among related classes, and an **interface to define a rule (contract)** that many unrelated classes can follow. In real projects both are often used together — an interface for the contract, and an abstract class that gives a common default code for it.

— End of Model Answers —

17 answers • all OR alternatives included • simple language edition
Nepal College of Information Technology | .NET Technologies (Elective)

Prepared by **Arpan Adhikari**

Revision order (most marks first):
1. **Diagrams** — architecture, page life cycle, DI, GC generations, monolith vs microservices
2. **Tables** — Framework vs Core vs .NET, lifetimes, status codes, abstract vs interface
3. **Definition boxes** — write this line first in every answer
4. **Bold keywords** — CLR, JIT, IL, ViewState, DbContext, 401/403, Gen 0/1/2
5. Code — only 5 to 8 lines is enough